

Functional Self-Awareness for Hierarchical SARSI Multi-Agent Systems: Version 3.1

Deployment Edition — What Building It Changed

An implementation-grounded revision of Version 3.0,
written from the console running at `singularity.comparegpt.io`

Chengshuai Yang

NextGen PlatformAI C Corp.

`spiritai@platformai.org`

July 30, 2026

Abstract

Version 3.0 [1] specifies a hierarchical SARSI architecture: manager and specialist self-states, nested workspaces, typed directives and reports, evidence-calibrated reliability, soft influence weighting, a class-based interrupt policy, and a local stability condition. This edition is not a restatement. It is the same architecture read back off a running system — the governed console deployed at `singularity.comparegpt.io` — and it records what building it changed.

The contribution is threefold. First, a **conformance map**: every equation and invariant of Version 3.0 is traced to the module, function, and test that implements it, or marked as not built. This is what makes the specification usable by someone extending the system rather than reading about it. Second, **seven implementation-derived revisions**. The most consequential is that the first fleet channel was built as a single shared workspace, which the paper’s own Invariant 4 forbids and which was corrected in place to a nested one — a defect the specification caught before deployment did. The others concern the reading of an unmeasured reliability, the difference between bounding a round and bounding storage, the consumption semantics of an admitted report, silent truncation, an over-computed softmax, and the observation that a specification which groups two interrupt classes into one branch has under-specified both. Third, an account of **invariants as executable objects**: in the deployed system the paper’s prohibitions are not prose but named tests, and the paper lists them by name so that a reader can check a claim rather than trust it.

Two divergences between corpus and runtime are reported rather than silently reconciled. The runtime ships four autonomy ceilings where the corpus describes five, and the highest deployed ceiling is *earned* through a verified trust ledger rather than set — a stronger form of the no-authority-through-influence invariant than the paper states. And the deployed metacognitive repertoire contains `recover` and omits `use tool`, which is not a naming difference: recovery acts on the agent’s own resource coordinate rather than on the task, and belongs in the repertoire for the same reason abstention does.

The paper closes with what is not built, in dependency order, and with the honest observation that the architecture’s most load-bearing guarantees are currently free: they hold because the channel that would violate them does not exist yet, not because it is policed.

Keywords: SARSI; multi-agent systems; functional self-awareness; global workspace; deployed systems; implementation report; agent governance; conformance.

Contents

1	Why This Edition Exists	3
1.1	The gap a specification cannot close by itself	3
1.2	What this edition is	3
1.3	What this edition is not	3
2	The Deployment	3
3	Conformance Map	4
4	Seven Revisions That Building Produced	5
5	Invariants as Executable Objects	6
6	Two Divergences Between Corpus and Runtime	7
7	What Is Not Built, In Dependency Order	8
8	Recommendations for Version 4.0	8
9	Conclusion	9

1 Why This Edition Exists

1.1 The gap a specification cannot close by itself

Version 3.0 is a complete architecture. It defines the manager and specialist states, the nested workspaces, the typed directive and report, the reliability posterior, the influence weighting, the interrupt classes, the safety invariants, and a sufficient stability condition, and it explains every equation symbol by symbol. What it cannot do, because no specification can, is say which of it is real.

That distinction matters more for this architecture than for most, because its central claims are prohibitions. A prohibition is satisfied trivially by a system that lacks the mechanism it forbids, and a reader cannot tell from the specification whether an invariant is enforced or merely unviolated. This edition separates the two everywhere it can.

1.2 What this edition is

It is Version 3.0 read back off the console deployed at `singularity.comparegpt.io`: a FastAPI service behind an authenticating gate, proxied by nginx, fronting a governed machine and manager runtime. The architecture is unchanged. What is added is the mapping from specification to implementation, the revisions that building produced, and an inventory of what remains unbuilt.

How to read this document

If you are *extending* the system, §3 is the map: equation, module, function, test. If you are *evaluating* the architecture, §4 is where the specification was wrong and §7 is where it is unrealised. If you are *auditing*, §5 lists the invariants as test names, so that every prohibition in this paper can be checked rather than believed.

1.3 What this edition is not

It does not revise the architecture’s mathematics. The state model, workspace equations, reliability posterior, influence weighting, and stability condition stand as Version 3.0 gives them, and where this paper cites an equation it cites that paper’s numbering.

It does not report benchmark results. The revisions below come from building and testing the system, not from measuring its performance against a baseline. Claims about what the architecture achieves in deployment are not made here, because the coupled system the architecture describes is not yet fully running (§7).

It does not describe the product. The console has a login, a session model, a billing path, and a user interface, none of which bear on the architecture and none of which appear below.

2 The Deployment

The system is a governed control plane for recursively self-improving agents, in which every learning agent is treated as an *untrusted proposer*: an agent may propose changes, while policy, evaluation, credentials, promotion, signing, audit, rollback, and shutdown remain external to it and owner-controlled. Task execution and self-improvement are separate loops behind a governance plane the agent cannot modify. That separation is the deployment’s version of Version 3.0’s proposal/evaluator invariant, and it predates it.

Three properties of the deployment shape everything below.

Table 1: The modules that carry the architecture. Line counts are indicative of scope, not quality.

Module	Lines	Role in Version 3.0’s terms
<code>fleet_workspace.py</code>	271	The manager/specialist channel: W_i^M , Proj_j , typed report, reliability, influence weight
<code>fleet.py</code>	363	Fleet composition and operator view
<code>manager.py</code>	643	Manager agent and delegation
<code>selfmodel.py</code>	392	Evidence-linked self-state; the seven-level authority ladder
<code>metacog.py</code>	365	Metacognitive action selection over the deployed repertoire
<code>governance.py</code>	162	Ceilings, trust ledger, machine pause, session control
<code>outcomes.py</code>	116	Verified-outcome store; competence and calibration posteriors
<code>evalstore.py</code>	1035	Evaluation store and console bench
<code>rsi.py, paramloop.py</code>	868	The slow structural-improvement loop
<code>workspace.py</code>	171	Single-agent bounded workspace

Authority lives outside the agent. Ceilings, the trust ledger, the pause flag, and session control are runtime state that the web layer reads and never extends. The console’s own documentation states that it “adds no authority of its own,” and where it could set a value the runtime would silently cap, it refuses loudly instead. That choice — refusing rather than silently capping — is a small thing that matters, because a silent cap teaches an operator that a control works when it does not.

Claims are derived at read time. The self-model is not a stored document that some process updates. It is computed on read from stores that already exist: the authoritative registry, the evaluation store, the runtime probes, and the narrative store. An empty store yields an honest “no verified outcomes recorded yet” rather than a default. This is the strongest available answer to confabulation, because there is no writable field to confabulate into.

The sandbox is the safety model. Rootless container isolation is a host requirement rather than a recommendation, and the installer refuses to proceed without it. The architecture’s action gating sits inside a boundary that does not depend on the agent respecting it.

3 Conformance Map

This is the section that makes the specification usable. Each row traces one object of Version 3.0 to the code that realises it and the test that holds it in place. Built means the mechanism exists and is exercised by a test; partial means it exists in reduced form, with the reduction named; not built means the specification stands unimplemented.

Reading the “not built” rows honestly

Five of fifteen rows are unimplemented, and they are not the easy five. The predictive dynamics, the interrupt state machine, and the algorithms are the parts that make the architecture a control system rather than a communication channel. What exists today is the channel plus the governance plane; what does not exist is the loop that runs on them. Section 7 gives the order in which they must arrive.

4 Seven Revisions That Building Produced

Revision 1 (A shared workspace is the wrong object, and the paper caught it).

The first version of the fleet channel built *one shared workspace*: every agent posted into it, every agent read from it, bounded by a per-agent cap and a set of reserved kinds. It was replaced in place.

The reason is Version 3.0’s own Invariant 4, which forbids raw cross-session broadcast: only typed summaries and referenced evidence may reach the manager, and a specialist may never see a peer’s raw report. A single shared channel violates that by construction, however carefully it is capped. The corrected module implements `Projj` as a filter — a specialist sees its own prior reports and manager-authored mission-level items, and nothing else.

This is worth recording precisely because it is the case where a specification earned its cost. The shared design was simpler, passed its own tests, and would have shipped. What caught it was a written invariant with a mechanical reading, and the correction was a fifteen-line function.

The general lesson

An invariant stated as prose is satisfied by whoever reads it charitably. An invariant with a named function that enforces it is satisfied by the system. The difference between Invariant 4 as written and `project_for_specialist()` as built is the difference between a rule and a mechanism, and only the second survives an implementer in a hurry.

Revision 2 (An unmeasured reliability is the prior mean, never zero).

Version 3.0’s Eq. (8) gives reliability as a Beta posterior with pseudo-counts $\alpha_0, \beta_0 > 0$, which already implies that a specialist with no verified outcomes sits at the prior rather than at zero. The implementation makes the consequence explicit, returning the prior mean when the verified count is zero, with the reason recorded in the code: *unmeasured is not zero*.

The reason to elevate this from an implication to a stated requirement is that the natural implementation is wrong. A developer writing a reliability function will reach for a ratio of successes to attempts, which is undefined at zero attempts and is then defensively coerced — almost always to zero, occasionally to one. Zero reads as *this specialist fails*, and a new specialist is then never ranked highly enough to earn the outcomes that would correct the estimate. One reads as *this specialist is perfect*, and it is trusted with exactly the work that should have been withheld. Both are self-fulfilling in opposite directions.

The Beta form is what makes the correct behaviour available, and the pseudo-counts are not a smoothing convenience. They are what stops a cold-start estimate from being an assertion.

Revision 3 (Bounding a round and bounding storage are different problems).

Version 3.0 gives a per-agent cap κ_a on admission to one workspace. That bounds *attention* in a round, and it is sufficient for the stated purpose: one specialist in a failure loop cannot monopolise the channel.

It does not bound *storage*. A specialist emitting an item per second accumulates unbounded pending reports whether or not any are admitted, and the cap that protects the manager’s attention does nothing for the disk. The implementation therefore carries three bounds where the paper carries one: a per-round admission cap, a per-specialist pending queue bound, and a time-to-live after which an unadmitted report expires unseen.

The third is the one that is easy to miss and unpleasant to omit. Without it, a burst during a busy period lingers in the queue and is admitted much later, during a quiet round, where it competes against current traffic on the strength of an urgency measured against conditions that no longer exist. Reports carry a timestamp, so staleness is checkable; nothing in the specification says to check it.

Revision 4 (An admitted report must be consumed).

Version 3.0 describes admission and does not describe what happens afterwards. If an admitted report remains pending, it is re-admitted every round for as long as it outranks the alternatives, and a single high-salience item becomes a permanent fixture of the manager’s attention.

The implementation consumes on admission: an incorporated report is removed from the queue. This is a one-line property with a large behavioural consequence, and it belongs in the specification because the alternative — a workspace that re-selects the same item indefinitely — looks correct in a unit test with one round and fails only under sustained operation.

Revision 5 (Nothing may be dropped silently).

A bounded channel discards. The specification bounds the channel and does not say what a discard obliges. The implementation returns the count of reports that did not fit alongside the ones that did, and the rendered workspace states it in words.

The reason is the same reason the self-model reports an empty store honestly. A truncated view that does not announce its truncation is read as complete, and a manager reasoning over the admitted items will treat their absence of contradiction as evidence of consensus. A dropped count is the minimum honest signal, and it costs one integer.

Revision 6 (The softmax need not be computed to be used).

Eq. (9) presents the influence weight as a softmax over the scored terms, which is correct as a definition of a normalised weight and unnecessary as a ranking procedure. The softmax is monotone in its logits, so ordering by the raw score gives the same order at lower cost, and the implementation ranks on the logit.

The distinction is not merely an optimisation. The normalised weights are needed where the aggregate matters — Version 3.0’s Eq. (10) sums $\alpha_{j,i} \Gamma_{jM} r_t^{j \rightarrow M}$, and there the convexity of the weights is what bounds the aggregate inside the convex hull of the projections. They are not needed where only the order matters. A specification that presents one expression for both uses invites an implementer to compute a normalisation and then discard it, or worse, to normalise across a set that changes between rounds and wonder why rankings move.

Revision 7 (Grouping two interrupt classes into one branch under-specifies both).

Version 3.0’s interrupt policy lists a branch covering *steer* and *revise-mission* together, both mapping to checkpoint-and-replan. The implementation notes refer to a six-class machine, counting them separately, and the two counts disagree because the classes genuinely differ.

Steering leaves the mission intact and changes how it is pursued: prior verdicts remain valid, the plan generation need not increment, and completed phases stay complete. Revising the mission invalidates verdicts that were conditioned on the old objective, requires a generation increment under the base paper’s own generation-consistency invariant, and may render completed phases irrelevant rather than complete. Mapping both to “checkpoint and replan” names the mechanism they share and hides the part that differs, which is what happens to work already done.

The recommendation is that the two be separate classes with separate semantics for verdict validity and generation increment, and that the count be six.

5 Invariants as Executable Objects

In the deployed system the architecture’s prohibitions are tests. This section lists them by name, because a prohibition that can be checked is a different kind of claim from one that must be believed, and a reader should be able to tell which they are being offered.

The eighth row deserves comment. The test named there is the mechanical form of the architecture’s most important prohibition, and it is currently satisfied structurally: the channel module has no import path that reaches a self-state write. The test asserts that this remains true, which is the right thing to assert, because the property is one refactor away from being lost and no reviewer would notice its absence.

A structural guarantee is not a policed one

Most of these tests currently pass because the forbidden mechanism does not exist rather than because an attempt to use it is rejected. That is a strong position while it lasts and a fragile one to rely on. When a directive channel is built (§7), the cross-write prohibition stops being structural and must become enforced — and the test above is what will fail first if it is not.

6 Two Divergences Between Corpus and Runtime

Revision 8 (The runtime ships four ceilings, and the highest is earned).

The corpus describes five autonomy ceilings. The runtime defines four, and the difference is not a truncation.

The deployed top ceiling is not a setting. It is unlocked by a trust ledger that counts verified approvals against a threshold and requires zero forbidden-operation trips; the console refuses to set it directly and calls the ledger’s own self-gating unlock instead. Authority at the top of the deployed lattice is therefore *earned through recorded history*, not granted.

This is a stronger mechanism than the specification describes, and it deserves to be lifted into the architecture rather than left as a deployment detail. The base paper’s invariant that influence cannot widen authority prevents a bad path to more authority. An earned ceiling supplies the good one, and it supplies it in exactly the currency the rest of the architecture uses: verified outcomes. A ceiling that can only be reached by accumulating approvals with no forbidden trips is an authority claim with provenance, which is what every other claim in this architecture is required to have.

The recommendation is to state the ceiling lattice as four levels with the top earned, and to specify the ledger’s inputs — approvals, denials, forbidden trips, threshold — as first-class governance state.

Revision 9 (The deployed repertoire has `recover` and lacks `use tool`).

The specification’s action repertoire is *act, use tool, verify, delegate, clarify, abstain, escalate*. The deployed set is *act, verify, delegate, clarify, abstain, escalate, recover*. Both have seven members; two differ.

Dropping `use tool` is the smaller change and is defensible: tool use is a way of acting rather than an alternative to it, and separating them creates a distinction the controller never needs to make. The specification can fold it into `act` without loss.

Adding `recover` is the substantive one. Recovery is the action an agent takes on *itself* — restoring a degraded session, repairing its own context or process state — rather than on the task. It belongs in the repertoire for precisely the reason abstention does: both are cases where the correct output is not progress on the objective, and a repertoire that cannot express them forces the controller to choose among actions that are all wrong. The deployed control code makes this explicit, selecting recovery-adjacent actions with reference to the agent’s resource coordinate directly.

That last detail is worth noting on its own. The control code names self-state coordinates when it decides. The self-model is not a reporting artefact bolted onto the side of the system; it is read by the thing that chooses the next action, which is the only arrangement in which functional self-awareness earns the word *functional*.

7 What Is Not Built, In Dependency Order

The unbuilt parts are the ones that turn a channel into a loop. They have a forced order, and the order is not the order in which they appear in Version 3.0.

1. **The typed directive.** Reports flow upward as typed objects; directives do not yet flow downward as any object at all. Nothing above this can be built without it, because a control loop needs both halves. It is also the point at which the cross-write prohibition stops being free: a directive channel is the mechanism that could carry a state write, so the enforcement must be written at the same time as the channel, not after.
2. **Recorded forecasts.** The predictive dynamics require that a forecast be durably written *before* the action it predicts. A forecast recorded afterwards is a postdiction and its residual teaches nothing. This is a storage-ordering requirement rather than a modelling one, and it must precede any residual-driven machinery.
3. **The interrupt state machine.** Pause and stop exist as governance controls. The class machine — with the six classes of Revision 7, the safe-point guarantee, and the no-new-side-effects interval — does not. It depends on the directive channel, because steering and mission revision are directives, and on nothing else.
4. **The coupled dynamics.** Only once directives, forecasts, and residuals exist is there a coupled system whose stability condition means anything. Until then the condition is a statement about an architecture rather than a property of a deployment, and reporting it as the latter would be a category error.
5. **Delegation between specialists.** Not implemented, which is why the intersection rule for delegated authority is unenforced. It should be written before delegation is, since retrofitting an intersection rule onto a live delegation graph means auditing every existing path rather than checking one function.

The honest status of the guarantees

The architecture's strongest claims are currently free. No agent writes another's self-state because there is no channel that could. Delegated authority cannot be laundered because there is no delegation. These are true statements about the running system and weak evidence about the architecture, and the distinction should be maintained in anything that cites this work. The specification's value so far has been in what it stopped (Revision 1), not in what it has been shown to guarantee.

8 Recommendations for Version 4.0

The revisions above are corrections to Version 3.0 as written. The following are changes of shape, offered for the next full edition rather than applied here.

Specify the storage discipline alongside the attention discipline. The per-round cap is one of three bounds a real channel needs (Revision 3), and the other two are not derivable from it. A specification that gives κ_a and stops has described a third of the problem.

Give every invariant a mechanical reading. Invariant 4 was enforceable because it could be turned into a filter over a projection. Not every invariant in the list has that property, and the ones that do not are the ones most likely to be honoured in the letter. Where an invariant cannot be reduced to a function, the specification should say so and name what would have to be audited instead.

Promote the trust ledger into the architecture. An earned top ceiling (Revision 8) is a mechanism the architecture wants and does not have. It closes the gap between forbidding a bad route to authority and providing a good one.

State the cold-start reading of every estimator. Reliability is not the only quantity with an undefined value at zero evidence; competence, calibration, and any developmental trend share the problem. The rule that unmeasured is the prior and never zero (Revision 2) should be stated once, globally, rather than rediscovered per estimator.

Separate the definitional form of a weight from its ranking use. Revision 6 is a small thing that will be got wrong repeatedly, because the specification currently presents one expression for two purposes with different requirements.

9 Conclusion

Version 3.0 specified a hierarchical architecture in enough detail to be built, and this edition reports what building it produced. The largest result is not any of the seven revisions individually but the pattern they share: every one of them concerns a place where the specification described a mechanism's purpose without describing its edges — what happens at zero evidence, what happens after admission, what happens to what does not fit, what happens to work already done when a mission changes. Those edges are where implementations diverge from specifications, and they are cheap to specify and expensive to discover.

The one case where the specification did its job unambiguously is Revision 1: a shared workspace was built, an invariant said it was wrong, and a nested one replaced it before deployment. That is the return on writing the architecture down, and it is worth more than the revisions cost.

The honest summary of the deployment is that the communication channel and the governance plane exist, the control loop does not, and the architecture's most quoted guarantees hold today because the mechanisms that would violate them have not been built. Section 7 gives the order in which that changes, and the first item on it — the typed directive — is also the one that converts the cross-write prohibition from a structural fact into something that must be enforced. That is the next thing to write, and it should be written together with its enforcement rather than before it.

Acknowledgments

This edition was produced by reading the deployed console against the architecture it implements. The revisions it records were found by building the system, and one of them was found by the specification before the system could ship it.

Conflict of Interest

The author is affiliated with NextGen PlatformAI C Corp., which develops SARSI systems and operates the deployment described here.

References

- [1] Chengshuai Yang. Functional self-awareness for hierarchical SARSI multi-agent systems: Version 3.0 — a brain-inspired manager–specialist architecture with bidirectional influence, nested workspaces, and safe user interruptibility. Preprint. Referred to throughout as “Version 3.0”, 2026.

Table 2: Version 3.0 to implementation. Equation numbers are that paper’s.

Specification	Implementation	Status	Note
Manager workspace W_i^M , Eq. (3)	<code>manager_ workspace()</code>	built	Hard classes reserved before ranking; routine ranked by influence weight
Specialist projection $\text{Proj}_j(W_i^M)$, Eq. (4)	<code>project_for_ specialist()</code>	built	Invariant 4 made mechanical rather than stated
Typed report $r_i^{j \rightarrow M}$, Eq. (7)	<code>report()</code>	built	Eight provenance fields; never raw text alone
Reliability $\rho_{j,t}$, Eq. (8)	<code>reliability()</code>	built	Reuses the verified-outcome posterior rather than adding a second competence store
Soft influence weight $\alpha_{j,t}$, Eq. (9)	<code>_softmax_weight()</code>	partial	Ranked on the logit; normalisation omitted (Revision 6)
Evidence-authority ladder	<code>selfmodel. AUTHORITY_NAMES</code>	partial	Seven levels defined, five populated
Self-state coordinates	<code>selfmodel, metacog</code>	partial	Referenced by name in control code, e.g. the resource coordinate on a stuck session
Action repertoire, Eq. (8) of base	<code>metacog.ACTIONS</code>	partial	Seven actions, differing in membership (Revision 9)
Authority chain	<code>governance.CEILINGS</code>	partial	Four ceilings, highest earned (Revision 8)
Promotion gate, Eq. (20) of base	<code>rsi.py, paramloop.py</code>	built	Proposer cannot modify evaluator, held-out set, or approval rule
Top-down directive $d_i^{M \rightarrow j}$, Eq. (6)	—	not built	Manager-to-specialist directive not yet a typed object
Predictive state dynamics, §8	—	not built	Explicitly out of scope in the channel module
Interrupt state machine, §9	<code>governance pause</code>	partial	Pause and stop exist; the class machine does not
Algorithms 1–3	—	not built	The channel is what they would sit on
Stability condition, Eq. (24)	—	not built	Requires the coupled dynamics; nothing to measure yet

Table 3: The fleet channel’s invariants, as named tests. Each corresponds to a claim made in Version 3.0 or in this edition.

Test	Claim it holds in place
<code>a_report_is_typed_not_raw_text</code>	Eq. (7): reports carry provenance
<code>reliability_is_a_verified_outcome_posterior</code>	Eq. (8); and Revision 2
<code>hard_class_reports_are_reserved_ahead_of_softmax_ranking</code>	An owner interrupt is not a term in an average
<code>routine_reports_compete_by_softmax_weight</code>	Eq. (9)
<code>the_manager_workspace_never_drops_anything_silently</code>	Revision 5
<code>a_specialist_projection_never_contains_a_peers_raw_report</code>	Invariant 4; Revision 1
<code>a_specialist_projection_keeps_its_own_reports_and_manager_facts</code>	Eq. (4): the projection is a filter, not a blackout
<code>broadcasting_never_writes_another_agents_self_state</code>	The cross-write prohibition
<code>a_report_once_admitted_is_not_rebroadcast_forever</code>	Revision 4
<code>a_floods_pending_queue_is_bounded_per_specialist</code>	Revision 3, storage bound
<code>stale_reports_expire_unseen</code>	Revision 3, time bound
<code>render_separates_reserved_from_ranked</code>	The reserved/ranked distinction survives presentation